

FLASH: Efficient Visuomotor Policy via Sparse Sampling

Jiaqi Bai*, Jindou Jia*, Yuxuan Hu, Gen Li, Xiangyu Chen, Tuo An, Kuangji Zuo, Jianfei Yang†

MARS Lab, Nanyang Technological University, Singapore

*Equal contribution.

†Corresponding author.

Generative models such as diffusion and flow matching have become dominant paradigms for visuomotor policy learning, yet their reliance on iterative denoising incurs high inference latency incompatible with real-time robotic control. We present **F**ast **L**egendre-polynomial **A**ction policy via **S**parse **H**istory-anchored flow (**FLASH** Policy), which replaces discrete action-chunk generation with continuous *Legendre* polynomial trajectory representation. Specifically, by fitting expert demonstrations under sparse temporal sampling, FLASH enables a single inference to cover a significantly extended action horizon. To further accelerate generation, FLASH initiates the flow matching process from history polynomial coefficients rather than uninformative *Gaussian* noise, shortening the transport distance and enabling accurate single-step inference. Moreover, analytic polynomial differentiation directly provides desired velocity feed-forward signals to the torque controller without numerical approximation. Extensive experiments on five simulated and two real-world manipulation tasks demonstrate that FLASH achieves state-of-the-art success rates ($\geq 92\%$ across all tasks), a per-episode inference time of 31.40 ms (up to $175\times$ faster than diffusion policies and $18\times$ faster than prior flow matching policies), up to $4\times$ faster training convergence than ACT, and $5\times$ to $7\times$ reduction in controller tracking error compared to discrete-action baselines.

Correspondence: Jianfei Yang (jianfei.yang@ntu.edu.sg), Jiaqi Bai (baij0018@e.ntu.edu.sg)

Project Page: <https://blue-jay.github.io/FLASH>



1 Introduction

Imitation learning has enabled robots to acquire complex manipulation skills through expert demonstrations. Recently, generative modeling such as diffusion policy (Chi et al., 2025) and flow matching (Lipman et al., 2023) has emerged as a dominant paradigm for policy representation in imitation learning, exhibiting exceptional multi-modal distribution modeling capabilities. However, their reliance on iterative denoising or ordinary differential equation (ODE) solving leads to high inference latency, making it difficult to satisfy real-time control requirements in robotic systems.

To mitigate inference latency, prevailing methods primarily aim to reduce the number of inference steps (Song et al., 2020a, 2023; Meng et al., 2023; Salimans and Ho, 2022; Sauer et al., 2024; Geng et al., 2025b; Jia et al., 2026). Despite these advancements, they still rely on discrete action-point generation that requires high-frequency inference over short action chunks, where each chunk typically spans only a short duration (e.g., ~ 0.8 seconds in π_0 (Black et al., 2024)). A natural approach is to increase the prediction horizon to reduce inference frequency and improve real-time efficiency. However, doing so for a single prediction inevitably leads to a substantial expansion in output dimensionality, resulting in the “curse of dimensionality” and significantly increased computational cost. This fundamental trade-off raises a key question: **Can low-frequency inference meet high-frequency control demands without sacrificing control fidelity or increasing model complexity?**

Our key observation is that robot motions are inherently smooth and low-frequency, and a simple *Legendre* polynomial can represent the discrete action points with only a few coefficients. While there exist recent works adopting polynomial-based representations for trajectory planning (Carvalho et al., 2025; Singha et al., 2026), they still use the original number of action points to fit the polynomial. To achieve highly efficient inference, we propose **F**ast **L**egendre-polynomial **A**ction policy via **S**parse **H**istory-anchored flow (**FLASH**) that incorporates sparsity and generation mode for polynomial policy learning. Firstly, a *sparse temporal sampling* strategy is proposed to fit polynomial coefficients at a significantly long temporal stride, as Fig. 1

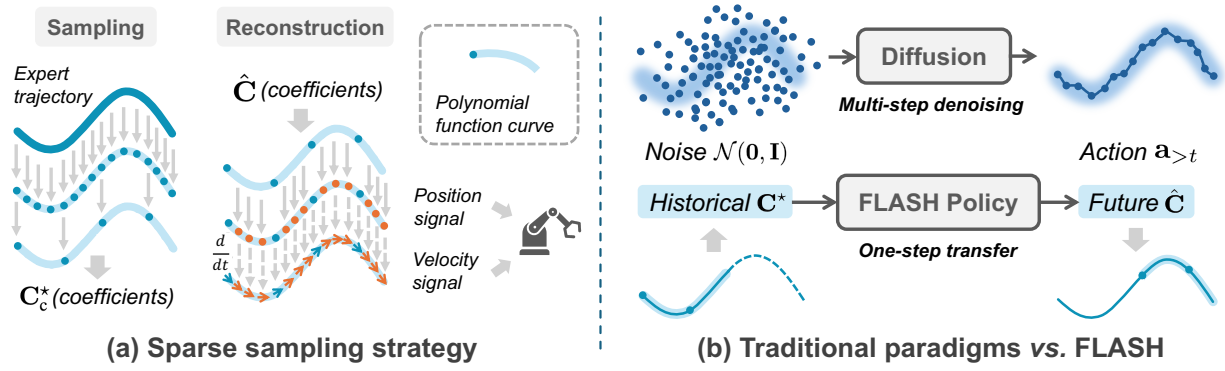


Figure 1 Overview of FLASH Policy. (a) **Sparse sampling strategy:** expert trajectories are fitted to polynomial coefficients $\mathbf{C}_c^*$ at a significant long temporal stride. During deployment, we perform dense temporal upsampling on the predicted coefficients $\hat{\mathbf{C}}$. Both position and velocity signals are fed to the controller. (b) **Traditional paradigms vs. FLASH:** conventional generative policies (top) generate discrete, non-smooth action points through multi-step denoising from uninformative noise. FLASH (bottom) operates in the coefficient space, transferring the historical coefficients to future coefficients with only one step, yielding a smooth continuous trajectory.

(a), so that each inference can cover a substantially extended action horizon without increasing model scale, directly answering the question above. Secondly, we introduce FLASH flow matching mechanism that initiates generation from the polynomial fitted to the measured action history, as Fig. 1 (b). This keeps the entire generative process in a compact, structured coefficient space, dramatically shortening the transport distance and enabling accurate single-step inference. Moreover, to alleviate discontinuities between adjacent polynomial chunks, *cross-horizon kinematic continuity* constraints are introduced to guarantee C^1 continuity (position and velocity) (Abramowitz and Stegun, 1948) at the transition points.

The polynomial representation offers additional benefits for low-level controllers. First, its differentiable nature enables analytic computation of desired velocity via simple differentiation, which we leverage as a feedforward signal to the low-level torque controller to substantially reduce end-effector tracking errors. Second, this representation fundamentally guarantees the spatial smoothness of the trajectory and aligns with the varying control frequencies across different robotic platforms, which is hardware-friendly to low-level controllers. Third, during inference, task execution can be accelerated or decelerated by simply adjusting the evaluation temporal stride, satisfying the specific timing constraints of different tasks. We thoroughly evaluate FLASH in both simulated environments and real-world robotic systems. The empirical results show that FLASH exhibits superior performance regarding the success rate, efficiency, controller tracking error, and smoothness, consistently outperforming nine state-of-the-art (SOTA) baselines.

To recap, the primary contributions of this work are as follows:

- We propose **F**ast **L**egendre-polynomial **A**ction policy via **S**parse **H**istory-anchored flow (**FLASH** Policy), which compresses a significantly long trajectory as only a few *Legendre* polynomial coefficients fitted under sparse temporal sampling. Meantime, FLASH initiates the generative flow from a history prior in the coefficient space, enabling accurate single-step inference.
- We further introduce *cross-horizon C^1 kinematic continuity* constraints for smooth chunk transitions, and leverage the differentiability of polynomials to feed the analytic velocity feed-forward signals to the torque controller without numerical approximation or additional training supervision.
- Extensive experiments across seven tasks demonstrate that FLASH achieves state-of-the-art success rates ($\geq 92\%$ across all tasks), a per-episode inference time of 31.40 ms (up to $175\times$ faster than diffusion policy (Chi et al., 2025) and $18\times$ faster than prior flow matching (Lipman et al., 2023)), up to $4\times$ faster training convergence than ACT (Zhao et al., 2023), and $5\times$ to $7\times$ controller tracking error reductions.

2 Related work

2.1 Generative visuomotor policies

Diffusion policy (Chi et al., 2025) first formulates visuomotor control as conditional denoising diffusion process (Ho et al., 2020), generating action chunks via iterative DDPM sampling. Flow matching (Lipman et al., 2023) offers a simulation-free alternative with more flexible ODE solvers. Subsequent efforts focus on accelerating diffusion and flow inference through faster samplers (DDIM (Song et al., 2020a), score matching (Song et al., 2020b)), consistency models (Song et al., 2023), diffusion distillation (Salimans and Ho, 2022; Meng et al., 2023; Sauer et al., 2024), MeanFlow (Geng et al., 2025b), more expressive architectures (DiT (Peebles and Xie, 2023)), and visual tokenization with flow-based generation (VITA (Gao et al., 2026)). While A2A (Jia et al., 2026) enables faster generation by replacing the uninformative *Gaussian* prior with the previous *action* chunk, it still operates in the discrete action space. Instead, FLASH operates in the *functional* level. The generative flow is initiated from the polynomial coefficients fitted to the measured action history, a trajectory-level prior that preserves richer kinematic structure than a flat action-chunk prior. More broadly, all the above methods predict fixed-length discrete action chunks that inherently suffer from inter-chunk jitter and require high-frequency re-inference. In contrast, FLASH lifts the representation to continuous polynomial coefficients, decoupling inference frequency from control frequency.

2.2 Polynomial trajectory representations

Recent learning-based methods have begun to adopt polynomial and spline parameterizations in robot trajectory planning. MPD (Carvalho et al., 2025) trains diffusion models over B-spline control points, employing analytically derived velocity and acceleration solely as smoothness cost functions during the denoising process. FlowMP (Nguyen et al., 2025) extends this idea to flow matching, jointly learning position, velocity, and acceleration fields from B-spline training data, which triples the supervision dimensionality and increases training overhead. BEAST (Zhou et al., 2025) employs B-splines purely as an efficient tokenization scheme for transformer-based imitation learning, without exploiting derivative information. Crowd-FM (Singha et al., 2026) parameterizes trajectories as *Bernstein* polynomials for mobile robot crowd navigation, yet does not feed the analytic velocity to any low-level controller. A common limitation across these works is that *none* actually leverages the analytic differentiability of the polynomial representation to supply velocity feedforward signals to a torque-level controller. FLASH fills this gap. It learns inherently smooth *Legendre*-coefficient trajectories with cross-horizon C^1 continuity, from which desired velocity is derived via analytical differentiation, without any additional training overhead.

3 Fast legendre-polynomial action policy via sparse history-anchored flow

We consider a visuomotor imitation-learning setting in which a policy maps historical observations to a future action trajectory. Let $\mathbf{a}_{\leq t} = \{\mathbf{a}_{t-T_o+1}, \dots, \mathbf{a}_t\}$ denote the history actions¹ observed up to step t with observation horizon T_o , $\mathbf{I}_{\leq t}$ denote the corresponding image stream, and $\mathbf{a}_{>t} = \{\mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+T_a}\}$ denote the generated actions with action horizon T_a . As illustrated in Fig. 1, FLASH departs from discrete action-chunk generation (Zhao et al., 2023): rather than learning T_a discrete action points, FLASH generates a small set of polynomial coefficients from which the entire continuous trajectory segment is analytically reconstructed. The key idea to improve inference efficiency is twofold. On the one hand, discrete action chunks are parameterized by compact polynomials (Sec. 3.1), decoupling the network’s output dimensionality from the prediction horizon. On the other hand, the generative flow is initiated from the polynomial coefficients fitted to the action history rather than from noise (Sec. 3.2), shortening the transport distance and enabling accurate single-step generation. Sec. 3.3 details the learning objective.

¹We assume history actions are observable, a common setting in manipulation tasks where a direct semantic correspondence exists between actions and proprioceptive states (e.g. joint angle, end-effector pose, and their differential forms).

3.1 Sparse polynomial parameterization

Legendre trajectory representation. Instead of generating a sequence of discrete samples, we parameterize the action trajectory as a continuous function of a normalized time variable $s \in [0, 1]$. Each action dimension is expanded on a set of orthogonal basis functions $\{\Phi_j\}_{j=0}^K$:

$$\mathbf{a}(s) = \sum_{j=0}^K \mathbf{c}_j \Phi_j(s), \quad s = \frac{\tau - \tau_{\text{start}}}{\tau_{\text{end}} - \tau_{\text{start}}} \in [0, 1], \quad (1)$$

where $\Phi_j(s) \in \mathbb{R}$ is a scalar basis function, $\mathbf{c}_j \in \mathbb{R}^{d_a}$ is the corresponding coefficient vector, K is the polynomial degree, d_a is the action dimensionality, and $\tau_{\text{start}}, \tau_{\text{end}}$ are the first and last timesteps of the segment. The polynomial spans H_{poly} discrete steps; in its simplest form $H_{\text{poly}} = T_a$, covering the full action horizon. We adopt the shifted *Legendre* polynomials (Abramowitz and Stegun, 1948; Szeg, 1939) as $\{\Phi_j\}$, which are defined through the *Bonnet* recurrence (Spiegel et al., 1968) ($n = 1, \dots, K-1$):

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x), \quad P_0(x) = 1, P_1(x) = x. \quad (2)$$

Evaluated at $x = 2s - 1$, the *Legendre* family is orthogonal on $[-1, 1]$. This orthogonality ensures a well-conditioned *Gram* matrix during data fitting and produces coefficient distributions with comparable magnitudes, which is critical for stable flow matching. With this continuous representation in place, we next describe how to fit the coefficients to expert demonstrations efficiently.

Sparse temporal sampling fitting. To extend the physical prediction horizon without increasing the network’s output dimensionality, we apply a sparse temporal sampling strategy to the expert demonstrations. We extract nodes from the expert trajectory using a temporal stride k , so that H_{poly} sparse fitting nodes effectively cover a physical time span of $k \times H_{\text{poly}}$ steps. Let $\mathbf{Q} \in \mathbb{R}^{H_{\text{poly}} \times d_a}$ be the stacked matrix of observed sparse actions, and $\mathbf{S} \in \mathbb{R}^{H_{\text{poly}} \times (K+1)}$ be the *Legendre* basis matrix evaluated at the corresponding discrete timesteps. The optimal polynomial coefficients $\mathbf{C}^*$ can be obtained in closed-form via ordinary least squares (Boyd and Vandenberghe, 2004; Weisberg, 2005) (OLS):

$$\mathbf{C}^* = (\mathbf{S}^\top \mathbf{S})^{-1} \mathbf{S}^\top \mathbf{Q}. \quad (3)$$

This strategy leverages the analytic continuity of polynomials to compactly represent a trajectory segment spanning a longer physical horizon.

Cross-horizon continuity and two-tier extension. Solving $\mathbf{C}^*$ independently for adjacent sub-segments introduces position or velocity discontinuities at the junctions. To enforce C^1 continuity, we extend the fitting window with overlapping regions on both sides of the execution interval and place exact kinematic anchors at the segment boundaries. A closed-form *KKT* correction (Mellinger and Kumar, 2011) then projects the unconstrained $\mathbf{C}^*$ onto the constraint manifold, yielding the corrected coefficients $\mathbf{C}_c^*$. We further append fit-padding steps to pull these anchors into the stable interior of the *Legendre* fitting window, preventing boundary sensitivity. The corrected coefficients are then row-wise scale-normalized to stabilize the flow-matching objective. Detailed derivations are provided in Appx. A. The above pipeline produces the training targets. We now describe how the predicted coefficients are decoded into trajectories at deployment.

Temporal upsampling and frequency decoupling. Since the policy learns a trajectory representation under sparse sampling, its single inference output spans the physical time of $k \times H_{\text{poly}}$ steps window. During deployment, we perform dense temporal upsampling on the predicted coefficients $\hat{\mathbf{C}}$, evaluating the executable trajectory $\hat{\mathbf{q}}(s_i)$ and its exact higher-order kinematic derivatives either at the original frequency used to collect the expert trajectories, or at any arbitrary high-frequency grid required by different low-level controllers:

$$\hat{\mathbf{a}}(s_i) = \sum_{j=0}^K \hat{\mathbf{c}}_j \Phi_j(s_i), \quad \hat{\mathbf{v}}(s_i) = \frac{1}{T} \sum_{j=0}^K \hat{\mathbf{c}}_j \Phi'_j(s_i), \quad (4)$$

where T denotes the physical time span of the polynomial segment, $T = k H_{\text{poly}} / f_{\text{expert}}$, with f_{expert} the sampling frequency of the original expert trajectories. This sparse sampling strategy drastically mitigates

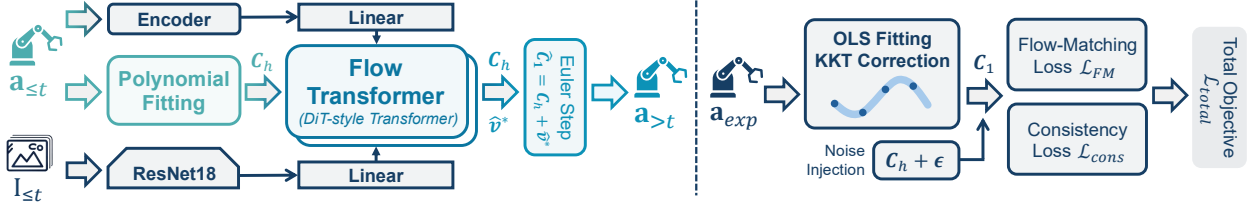


Figure 2 Overview of FLASH’s pipeline. Left: Inference. History actions are fitted into Legendre-polynomial coefficients and fused with visual features by a DiT-style Flow Transformer. A single Euler step predicts future coefficients, which are decoded into executable actions. **Right: Training.** Expert actions are converted into target polynomial coefficients via OLS fitting and KKT correction, and the model is optimized with flow-matching and consistency losses.

inference latency (by $1/k$) while using extended chunk lengths to smooth transitions at splicing boundaries. Moreover, the continuous polynomial representation naturally supports post-hoc temporal scaling: a single inference produces coefficients $\hat{\mathbf{C}}$ that fix the trajectory shape on the normalized time $s \in [0, 1]$, while the physical duration T over which this shape is played satisfies $T \propto k$ (Eq. (4)); the velocity command $\hat{\mathbf{v}} \propto 1/T$ is automatically rescaled by the same analytic derivative. Hence, simply setting the evaluation stride k_{eval} to a value different from the training stride k_{train} allows the same generated polynomial to be executed at any desired speed, with no retraining and no extra computation. This enables online acceleration of simple, safe sub-tasks for shorter execution time and online deceleration of high-precision, high-risk maneuvers. The achievable speed range is empirically validated in Sec. 4.4.

3.2 History-anchored flow

With the polynomial representation, we employ a flow-matching formulation to predict the transition of coefficients from adjacent history to the future timesteps, conditioned on the observations.

Flow matching. Flow matching (Lipman et al., 2023) learns a time-dependent velocity field f_θ that transports a source distribution p_0 to a target distribution p_1 along the probability path induced by the linear interpolant $\mathbf{a}_\tau = (1 - \tau)\mathbf{a}_0 + \tau\mathbf{a}_1$, $\tau \in [0, 1]$. The corresponding conditional velocity is $\mathbf{v}^* = \mathbf{a}_1 - \mathbf{a}_0$, and training minimizes $\mathbb{E}_{\tau, \mathbf{a}_0, \mathbf{a}_1} \|f_\theta(\mathbf{a}_\tau, \tau, \mathbf{e}) - \mathbf{v}^*\|^2$, where τ is the flow time uniformly sampled from $[0, 1]$ and $\mathbf{e}$ denotes the conditioning observation embedding. Standard flow matching policies fix $p_0 = \mathcal{N}(\mathbf{0}, \mathbf{I})$, which obliges the network to learn long transport paths and motivates the many-step ODE integration used at inference.

History-anchored flow. In visuomotor control, consecutive action segments share strong kinematic structure (Jia et al., 2026). We leverage this informative prior by defining the generative flow directly between history and future polynomials, rather than originating from uninformative *Gaussian* noise. Specifically, we fit the observed proprioceptive history $\mathbf{a}_{\leq t}$ to the *Legendre* basis to obtain the history coefficients $\mathbf{C}_h$. We use the same scheme as Eq. (3), but introduce *Tikhonov* regularization $\lambda_h > 0$. This formulation not only allows for fitting when the history window is configured to be short, but crucially, it mitigates the numerical ill-conditioning caused by dense discrete sampling over a narrow time horizon. By preventing coefficient explosion, it ensures a numerically stable and bounded starting distribution for the subsequent flow matching:

$$\mathbf{C}_h = (\mathbf{S}_h^\top \mathbf{S}_h + \lambda_h \mathbf{I})^{-1} \mathbf{S}_h^\top \mathbf{a}_{\leq t}. \quad (5)$$

FLASH then constructs a straight-line flow path from $\mathbf{C}_h$ to the target future coefficients $\mathbf{C}_1$:

$$\mathbf{C}_\tau = (1 - \tau) \mathbf{C}_h + \tau \mathbf{C}_1, \quad \mathbf{v}^*(\mathbf{C}_\tau, \tau) = \mathbf{C}_1 - \mathbf{C}_h. \quad (6)$$

A DiT-style transformer (Peebles and Xie, 2023) $f_\theta(\mathbf{C}_\tau, \tau, \mathbf{e})$ regresses this constant velocity $\mathbf{v}^*$, where the global condition $\mathbf{e}$ is obtained by encoding the multi-view image stream $\mathbf{I}_{\leq t}$ with a ResNet-18 encoder (He et al., 2016), concatenating each frame’s visual features with the corresponding proprioceptive history $\mathbf{a}_{\leq t}$, and flattening across the T_o observation steps, as shown in Fig. 2. Because $\mathbf{C}_h$ is typically close to $\mathbf{C}_1$, the transporting distance is short and a single *Euler* step $\hat{\mathbf{C}}_1 = \mathbf{C}_h + f_\theta(\mathbf{C}_h, 0, \mathbf{e})$ is sufficient at inference. The resulting $\hat{\mathbf{C}}_1$ is then decoded into the executable action trajectory $\hat{\mathbf{a}}_{> t}$ together with its analytical velocity profile via Eq. (4). During training, we inject minimal *Gaussian* noise $\epsilon \sim \mathcal{N}(\mathbf{0}, \sigma_h^2 \mathbf{I})$ into $\mathbf{C}_h$ to improve robustness against off-policy histories at deployment. With the parameterization (Sec. (3.1)) and flow architecture defined, we now formalize the learning objectives.

3.3 Learning objectives

Flow-matching loss. Following Eq. (6), the network is trained to regress the constant transport velocity along every intermediate point of the polynomial flow,

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\tau \sim \mathcal{U}[0,1], \mathbf{C}_h, \mathbf{C}_1} \left\| f_{\theta}(\mathbf{C}_{\tau}, \tau, \mathbf{e}) - (\mathbf{C}_1 - \mathbf{C}_h) \right\|_2^2. \quad (7)$$

Polynomial consistency loss. Eq. (7) supervises the vector field at arbitrary τ , but does not explicitly constrain the *one-step Euler* update used at inference. We therefore add a consistency term that enforces transport of $\mathbf{C}_h$ to $\mathbf{C}_1$ in a single evaluation of f_{θ} at $\tau = 0$:

$$\mathcal{L}_{\text{cons}} = \mathbb{E}_{\mathbf{C}_h, \mathbf{C}_1} \left\| \mathbf{C}_h + f_{\theta}(\mathbf{C}_h, 0, \mathbf{e}) - \mathbf{C}_1 \right\|_2^2. \quad (8)$$

This term bridges the gap between the continuous-time flow-matching objective and the discrete one-step ODE solver used at deployment, and is indispensable for retaining accuracy in the $N_{\text{NFE}} = 1$ regime (See ablation study in Sec. 5).

Total objective. With a scalar weight $\lambda_{\text{cons}} > 0$, the training loss reads

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{FM}} + \lambda_{\text{cons}} \mathcal{L}_{\text{cons}}. \quad (9)$$

All parameters of the vision encoder, condition projector, and flow transformer f_{θ} are optimized jointly. The scale-normalization matrix, least-squares solver (Eq. (3)), *Tikhonov*-regularized history solver (Eq. (5)), and *KKT* correction (Eq. (11)) are all precomputed and remain fixed during training.

4 Experiments

To evaluate the proposed method, we conduct the experiments in both simulated and real-world environments. As illustrated in Fig. 3, the evaluation encompasses seven manipulation tasks on *Franka* robot, five of which are in simulation on *Roboverse* (Geng et al., 2025a) platform, and two are real-world tasks.

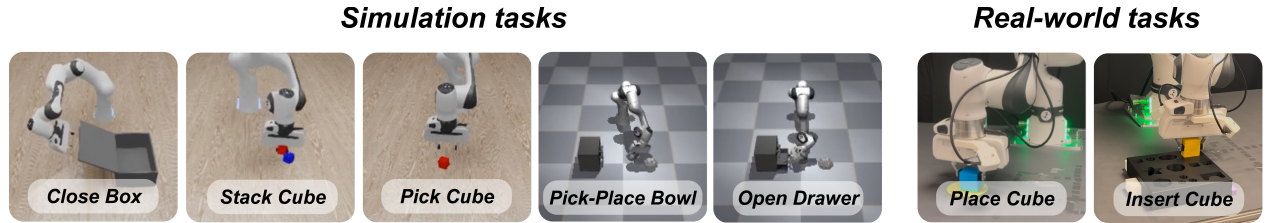


Figure 3 Left: Five simulated tasks from *Roboverse* (Geng et al., 2025a) platform. Right: Two real-world tasks.

For systematical comparison, we select nine state-of-the-art baselines: DDPM-UNet (Chi et al., 2025; Ho et al., 2020), DDPM-DiT (Ho et al., 2020; Peebles and Xie, 2023), DDIM-UNet (Chi et al., 2025; Song et al., 2020a), FM-UNet (Lipman et al., 2023), FM-DiT (Lipman et al., 2023; Peebles and Xie, 2023), Score-UNet (Song et al., 2020b), ACT (Zhao et al., 2023), VITA (Gao et al., 2026), A2A-Noise (Jia et al., 2026), and FLASH-Gaussian (FLASH-G). Specifically, FLASH-G is developed to serve as our homologous control group. It perfectly shares the network architecture for polynomials, *Legendre* basis and sparse sampling training strategy with FLASH. The only difference is that FLASH-G initiates flow matching from pure *Gaussian* noise ($p_0 = \mathcal{N}(0, \mathbf{I})$), representing the standard generation from noise paradigm similar to traditional generative paradigm. Therefore, the performance gap between FLASH and FLASH-G strictly isolates the independent contribution of the proposed “history-anchored flow” mechanism. Task descriptions, demonstration counts, and all hyperparameters are detailed in Appx. B.

As shown in Tab. 1, FLASH achieves $\geq 92\%$ success rate across all five tasks with single-step inference (NFE=1), outperforming the homologous FLASH-G by 17.6 pp on average (isolating the FLASH mechanism’s

Table 1 Success rates (%) on five tasks at a shared training budget of 10k optimizer steps. **NFE** denotes the number of generator function sampling steps at inference. Best results are **bold** while second-best results are underlined.

Method	NFE	Close Box	Pick Cube	Stack Cube	Open Drawer	Pick-Place Bowl
Score-UNet	100	44	58	24	2	8
DDPM-UNet	100	84	68	44	76	90
DDIM-UNet	40	80	74	42	76	<u>92</u>
DDPM-DiT	100	68	78	36	50	76
FM-DiT	10	70	92	46	36	68
FM-UNet	10	78	82	30	70	76
ACT	1	70	98	20	80	0
VITA	6	94	86	86	68	86
A2A-Noise	1	<u>98</u>	90	<u>92</u>	<u>88</u>	<u>92</u>
FLASH-G	10	70	94	82	62	88
FLASH (ours)	1	100	98	96	92	98

contribution) and the strongest single-step baseline A2A-Noise by 4.8 pp on average. Each entry is computed over 50 independent rollouts; at the median success rate of 92%, the 95% Clopper-Pearson confidence interval half-width is ± 7.5 pp. Details are provided in Appx. C.

Next, we provide a detailed statistical comparison from three perspectives: training efficiency (Sec. 4.1), inference speed (Sec. 4.2), and controller tracking error (Sec. 4.3). We also explore the possibility of adjusting the task execution speed through post-hoc temporal scaling (Sec. 4.4).

4.1 Training efficiency

Beyond a higher performance ceiling, FLASH exhibits significantly faster convergence. We evaluated the 11 policies across seven checkpoints at $\{2.5, 3.75, 5, 6.25, 7.5, 8.75, 10\} \times 10^3$ training steps under identical configurations. Fig. 4 illustrates the convergence speed on three representative tasks: *Pick Cube*, *Stack Cube*, and *Pick-Place Bowl* (results for other tasks are provided in Appx. D).

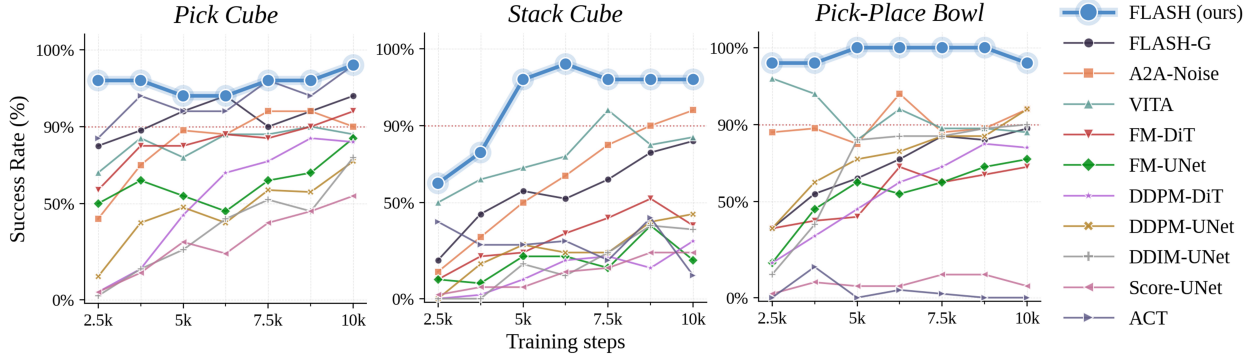


Figure 4 Training efficiency. Success rate versus training steps of 11 policies for three representative tasks.

It can be seen in *Pick Cube* task that FLASH reaches a 96% success rate in just 2,500 steps, whereas the strongest baseline, ACT (Zhao et al., 2023), requires 10,000 steps to reach a comparable level, making FLASH approximately $4\times$ faster to train. In *Stack Cube* task, FLASH achieves 98% success at 6,250 steps, while the most competitive baseline at that point (VITA (Gao et al., 2026)) sits at only 72%, a gap of nearly 30 percentage points. In *Pick-Place Bowl* task, FLASH stabilizes at a perfect 100% success rate after 5,000 steps, consistently outperforming all baselines throughout the remainder of the training. These rapid convergence properties demonstrate that operating in the history-anchored polynomial space drastically reduces the sample complexity and optimization burden compared to standard noise-to-action or discrete-action paradigms, allowing the policy to rapidly acquire robust behaviors.

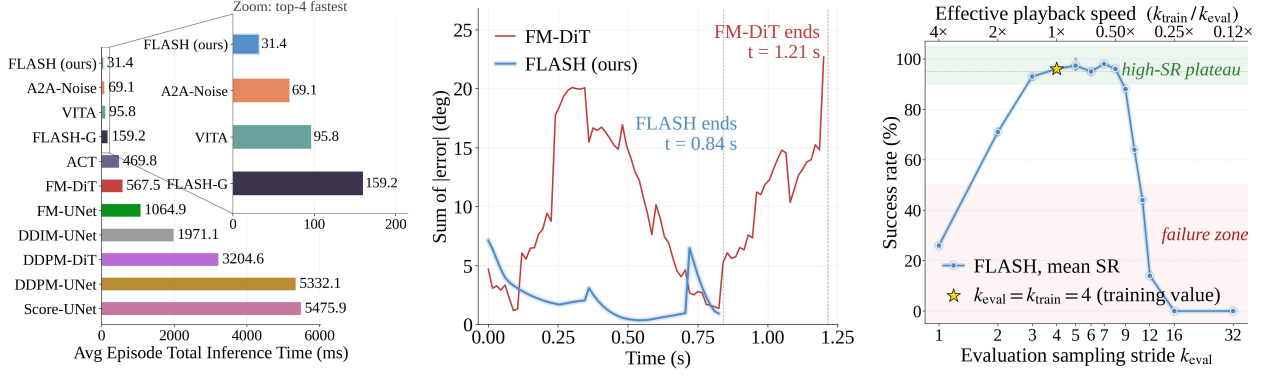


Figure 5 Simulation experiments. **Left:** total episode inference time on *Stack Cube* (all 11 policies, RTX 5090); inset zooms the top-4 fastest. **Middle:** sum of 7-joint absolute tracking errors on *Pick Cube*; dashed lines mark task completion times. **Right:** post-hoc speed modulation on *Stack Cube* by sweeping k_{eval} .

4.2 Inference speed

Inference speed is evaluated across three progressive dimensions: (a) total episode inference time, (b) per-call latency, and (c) real-world task completion wall time. For fair comparison, all the selected policies are employed on the same PC with NVIDIA RTX 5090 GPU. In this part, we mainly display the ablation results on (a), and the results on (b) and (c) are detailed in Appx. E.

As shown in Fig. 5 (left), this metric accumulates the duration of all inference calls within a successful rollout, directly reflecting the total computational cost required to complete a task. FLASH ranks first at 31.4 ms (averaged over all successful episodes): 175 $\times$ faster than Score-UNet (5476 ms), a difference of over two orders of magnitude; 2.2 $\times$ and 5.1 $\times$ faster than the strongest A2A-Noise (69 ms) and the homologous FLASH-G (159 ms). FLASH’s speed advantage stems from two independent mechanisms: (i) sparse sampling training (with a temporal stride $k = 4$), where each polynomial covers 4 $\times$ the action horizon of discrete policies, lowering the number of inference calls per episode; and (ii) the polynomial-to-polynomial flow starting from an informative history prior rather than noise, enabling single-step generation (NFE=1). The 5.1 $\times$ episode speedup over FLASH-G (which shares sparse sampling but starts from noise) isolates the contribution of this mechanism. See rigorous pairwise decomposition in Appx. E.

4.3 Controller tracking error

The polynomial representation in FLASH also benefits the low-level controller through analytic velocity feed-forward (Eq. 4) and native high-frequency signal generation. We validate this on both simulation and real-world tasks.

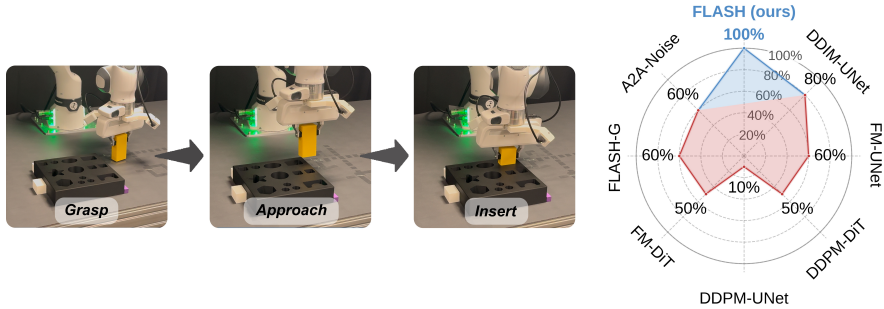


Figure 6 Real-world Insert Cube task. Execution snapshots (grasp $\rightarrow$ approach $\rightarrow$ insert) and success-rate radar chart across 8 policies under millimeter-level insertion tolerance.

In simulation (Fig. 5, middle), FLASH’s mean and peak tracking errors are 4.70 $\times$ and 3.18 $\times$ lower than FM-DiT, whose error curve spikes at every chunk boundary. On the real-world *Insert Cube* task with millimeter-level tolerance (Fig. 6), FLASH achieves 100% success rate, leading the other 7 baselines by an average of 47%. These results, together with the real-world tracking MAE analysis ($0.274 \pm 0.004^\circ$ for FLASH

vs. $0.460 \pm 0.028^\circ$ for FM-DiT over 5 rollouts in Appx. F, confirm that control-level precision is jointly improved by three advantages of polynomial parameterization: trajectory smoothness, native high-frequency control output, and analytic velocity feed-forward. Detailed analysis is provided in Appx. F.

4.4 Post-hoc execution speed modulation

Since the polynomial fixes the trajectory shape on a normalized time axis, execution speed can be modulated post-hoc by simply varying the evaluation stride k_{eval} relative to k_{train} , with no retraining. As shown in Fig. 5 (right), sweeping k_{eval} on *Stack Cube* reveals a wide high-success-rate plateau ($\geq 93\%$) spanning $0.5\times$ to $1.33\times$ playback speed. A detailed analysis of the failure modes at extreme speeds is provided in Appx. G.

5 Ablation study

To evaluate the contribution of each component, we conduct an ablation study on four key design choices in FLASH. Full experimental details and extended analysis for each ablation are provided in Appx. H.

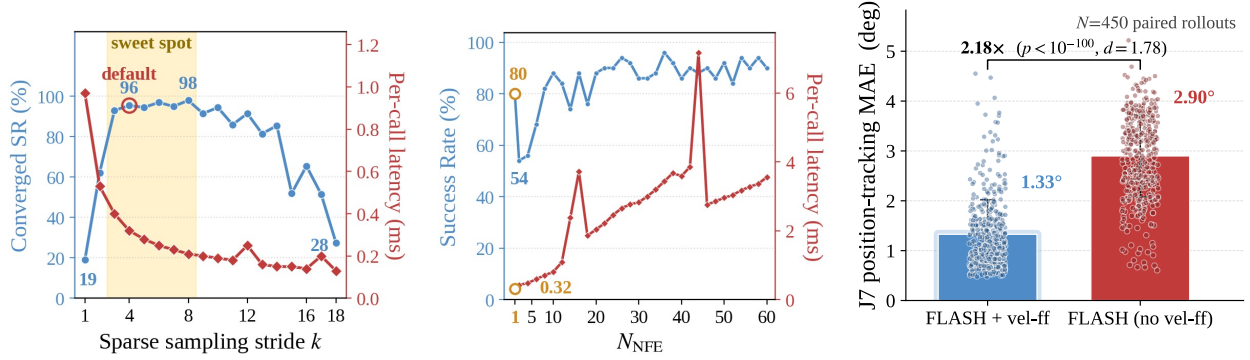


Figure 7 Ablation studies. **Left:** effect of sparse sampling stride k on *Stack Cube*; yellow band marks the sweet spot $k \in [3, 8]$. **Middle:** N_{NFE} sweep on *Close Box*; latency (red) grows linearly. **Right:** velocity feed-forward ablation.

Sparse sampling stride k . Sweeping k from 1 to 18 on *Stack Cube* (Fig. 7, left) reveals that sparse sampling is essential: $k=1$ caps at 24% success, while $k=4$ reaches 96% (+72 pp). A sweet spot at $k \in [3, 8]$ yields $\geq 93\%$ with monotonically decreasing latency; beyond $k \geq 13$ performance collapses due to loss of polynomial expressivity and insufficient visual feedback.

Fit padding and cross-horizon continuity. On the *Close Box* task, removing both fit padding (Eq. (12)) and KKT continuity (Eq. (11)) yields only 75.0%. Adding fit padding alone recovers 86.0% (+11.0 pp); further stacking the KKT C^1 correction (Eq. (11)) reaches 97.5% (+11.5 pp). Both mechanisms are necessary for the polynomial pipeline (see Tab. 2 in Appx. H.2 for details).

Number of function evaluations N_{NFE} . Single-step Euler ($N_{\text{NFE}} = 1$) achieves 80% success, *outperforming* multi-step integration at $N_{\text{NFE}} \in \{2, 4, 6\}$ (Fig. 7, middle). This validates the consistency loss $\mathcal{L}_{\text{cons}}$ (Eq. (8)), which specializes the model for single-step generation. Since latency grows linearly with N_{NFE} , any $N_{\text{NFE}} > 1$ is a strict efficiency loss with no accuracy upside.

Velocity feed-forward. Removing analytic velocity feed-forward inflates the J7 tracking MAE from 1.33° to 2.90° ($2.18\times$ worse; $p < 10^{-100}$, $N = 450$ rollouts; Fig. 7, right). Six of seven joints follow the same trend, confirming the feed-forward’s critical role.

6 Conclusion

We presented FLASH, a generative visuomotor policy that represents trajectories as *Legendre* polynomial coefficients. Three synergistic mechanisms: sparse temporal sampling, history-anchored flow matching, and analytic velocity feed-forward via polynomial differentiation, jointly enable single-step inference covering a

multi-fold extended action horizon, and precise torque-level control. Experiments on five simulated and two real-world tasks confirm state-of-the-art success rates ($\geq 92\%$), up to $175\times$ faster inference, $4\times$ faster training convergence and $5\times$ – $7\times$ tracking error reductions than prior state-of-the-art policies.

7 Limitations

Two key design parameters currently lack adaptivity. First, the polynomial degree K must be fixed before training and cannot be adjusted at inference time. The $K=6$ used throughout may be insufficient for trajectories with sharp, high-frequency components (e.g., contact-rich assembly or dexterous in-hand manipulation). Second, while the execution speed can be freely chosen at inference time by varying k_{eval} , it remains constant throughout a rollout and cannot adapt on the fly to task dynamics (e.g., slowing near a tight insertion while accelerating through free space). A promising future direction is to make both parameters *adaptive*: learning to predict a suitable polynomial degree per segment and to modulate execution speed in a closed loop.

References

- Milton Abramowitz and Irene A Stegun. *Handbook of mathematical functions with formulas, graphs, and mathematical tables*, volume 55. US Government printing office, 1948.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Stephen Boyd and Lieven Vandenbergh. *Convex optimization*. Cambridge university press, 2004.
- Joao Carvalho, An T Le, Piotr Kicki, Dorothea Koert, and Jan Peters. Motion planning diffusion: Learning and adapting robot motion planning with diffusion models. *IEEE Transactions on Robotics*, 2025.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- Jacob Cohen. *Statistical power analysis for the behavioral sciences*. routledge, 2013.
- Dechen Gao, BOQI ZHAO, Andrew Lee, Ian Chuang, Hanchu Zhou, Hang Wang, Zhe Zhao, Junshan Zhang, and Iman Soltani. VITA: Vision-to-action flow matching policy. In *Proceedings of the International Conference on Learning Representations*, 2026.
- Haoran Geng, Feishi Wang, Songlin Wei, Yuyang Li, Bangjun Wang, Boshi An, Charlie Tianyue Cheng, Haozhe Lou, Peihao Li, Yen-Jen Wang, et al. Roboverse: Towards a unified platform, dataset and benchmark for scalable and generalizable robot learning. *arXiv preprint arXiv:2504.18904*, 2025a.
- Zhengyang Geng, Mingyang Deng, Xingjian Bai, J Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. *arXiv preprint arXiv:2505.13447*, 2025b.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Stephen James, Zicong Ma, David Rovick Arrojo, and Andrew J Davison. Rlbench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 5(2):3019–3026, 2020.
- Jindou Jia, Gen Li, Xiangyu Chen, Tuo An, Yuxuan Hu, Jingliang Li, Xinying Guo, and Jianfei Yang. Action-to-action flow matching. *arXiv preprint arXiv:2602.07322*, 2026.
- Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *Proceedings of International Conference on Learning Representations*, 2023.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- Daniel Mellinger and Vijay Kumar. Minimum snap trajectory generation and control for quadrotors. In *Proceedings of IEEE International Conference on Robotics and Automation*, pages 2520–2525. Ieee, 2011.
- Chenlin Meng, Robin Rombach, Ruiqi Gao, Diederik Kingma, Stefano Ermon, Jonathan Ho, and Tim Salimans. On distillation of guided diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14297–14306, 2023.
- Tongzhou Mu, Zhan Ling, Fanbo Xiang, Derek Yang, Xuanlin Li, Stone Tao, Zhiao Huang, Zhiwei Jia, and Hao Su. Maniskill: Generalizable manipulation skill benchmark with large-scale demonstrations. *arXiv preprint arXiv:2107.14483*, 2021.
- Khang Nguyen, An T Le, Tien Pham, Manfred Huber, Jan Peters, and Minh Nhat Vu. Flowmp: Learning motion fields for robot planning with conditional flow matching. In *Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 11291–11297. IEEE, 2025.
- William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4195–4205, 2023.

- Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. *arXiv preprint arXiv:2202.00512*, 2022.
- Axel Sauer, Dominik Lorenz, Andreas Blattmann, and Robin Rombach. Adversarial diffusion distillation. In *Proceedings of European Conference on Computer Vision*, pages 87–103. Springer, 2024.
- Antareep Singha, Laksh Nanwani, Samkit Jain, Phani Teja Singamaneni, Arun Kumar Singh, K Madhava Krishna, et al. Crowd-fm: Learned optimal selection of conditional flow matching-generated trajectories for crowd navigation. *arXiv preprint arXiv:2602.06698*, 2026.
- Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020a.
- Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020b.
- Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *Proceedings of International Conference on Machine Learning*, 2023.
- Murray R. Spiegel, Seymour Lipschutz, and John Liu. *Mathematical Handbook of Formulas and Tables*. Schaum’s Outline Series. McGraw-Hill, New York, 1968. ISBN 978-0070380578.
- Gabor Szeg. *Orthogonal polynomials*, volume 23. American Mathematical Soc., 1939.
- Sanford Weisberg. *Applied linear regression*, volume 528. John Wiley & Sons, 2005.
- Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- Hongyi Zhou, Weiran Liao, Xi Huang, Yucheng Tang, Fabian Otto, Xiaogang Jia, Xinkai Jiang, Simon Hilber, Ge Li, Qian Wang, et al. Beast: Efficient tokenization of b-splines encoded action sequences for imitation learning. *arXiv preprint arXiv:2506.06072*, 2025.

Appendix

A Cross-horizon continuity and two-tier extension

This section provides the full derivation of the cross-horizon continuity mechanism summarized in Section 3.1.

First extension: overlap and exact anchors. The predicted polynomial extends beyond the execution interval of length T_a by μ_{pre} steps backward and μ_{post} steps forward. This forms an extended evaluation window of total length H_{poly} , representing the actual horizon covered by the predicted polynomial:

$$H_{\text{poly}} = \mu_{\text{pre}} + T_a + \mu_{\text{post}}. \quad (10)$$

These $\mu_{\text{pre}} + \mu_{\text{post}}$ overlap steps are supervised during training, but discarded before deployment. Within this window, we designate two constraint anchors exactly at the physical boundaries of the execution segment (the front anchor at $t + 1$ and the rear anchor at $t + T_a + 1$). At these anchors, we explicitly enforce the current polynomial’s kinematic states to match the expert trajectory.

This formulation supports arbitrary continuity up to C^r (e.g., C^3 for position, velocity, acceleration, and jerk) by appending exact derivatives to the constraints. Stacking these constraints ($2 \text{ anchors} \times \{r + 1\}$) into a matrix $\mathbf{A}$ with target $\mathbf{b}$, the closed-form *KKT* solution (Mellinger and Kumar, 2011) corrects the coefficients:

$$\mathbf{C}_c^* = \mathbf{C}^* - (\mathbf{S}^\top \mathbf{S})^{-1} \mathbf{A}^\top [\mathbf{A} (\mathbf{S}^\top \mathbf{S})^{-1} \mathbf{A}^\top]^{-1} (\mathbf{A} \mathbf{C}^* - \mathbf{b}). \quad (11)$$

Second extension: fit padding. For *Legendre* bases, the functions reach extreme values at the boundaries, making the least-squares solution highly sensitive to target perturbations. To prevent degradation of the fit quality near the anchors, we append δ padding steps to both sides, expanding the full regression window:

$$H_{\text{poly}} = \delta + \mu_{\text{pre}} + T_a + \mu_{\text{post}} + \delta. \quad (12)$$

These 2δ steps serve purely as a data-preprocessing method. They act as additional regression observation points to pull the constraint anchors into the stable interior of the fitting window, without receiving direct training supervision or being output by the network.

B Evaluation setup and details

Tasks and demonstrations. For the simulation experiments, we evaluate FLASH on five tasks: *Close Box* from RL Bench (James et al., 2020), *Stack Cube* and *Pick Cube* from ManiSkill (Mu et al., 2021) (using the Isaac simulator), as well as *Pick-Place Bowl* and *Open Drawer* from LIBERO (Liu et al., 2023) (using the MuJoCo simulator). We collected 100 expert demonstrations for *Close Box*, *Pick Cube* and *Stack Cube*, and 40 demonstrations for *Pick-Place Bowl* and *Open Drawer*. All models are trained for 10k optimizer steps and evaluated over 50 rollouts with randomized initial states.

Hyperparameters. To ensure a fair comparison, all methods share the same visual encoder (ResNet-18 (He et al., 2016) with group normalization), observation horizons, batch size, and optimizer schedule; the only varying factors are the algorithmic core and method-specific settings. Through extensive hyperparameter search, we find that A2A-Noise achieves its best performance with a noise scale of $\epsilon = 0.02$ and a single-step (NFE = 1) inference, which outperforms the original A2A with 6-step inference. We therefore report results with this best-performing configuration. For FLASH and FLASH-G, we use temporal sampling stride $k = 4$, Legendre degree $K = 6$, and FLASH’s unique setting is: a single-step Euler solver, the noise scale injected to the history $\sigma_h = 0.5$, *Tikhonov* regularization term $\lambda_h = 0.1$, and the weight for consistency loss $\lambda_{\text{cons}} = 1.0$.

C Success rate analysis

Table 1 presents the success rates of all 11 policies across the five simulated tasks under a 10k-step training budget. The empirical results demonstrate a three-tier progression:

- State-of-the-art performance:** FLASH ranks first on 4 out of 5 tasks, ties for first with ACT on *Pick Cube*, and is the only method to achieve $\geq 92\%$ success rate across all five tasks, requiring only a single sampling step (NFE=1) during inference.
- Significant improvement over FLASH-G:** FLASH outperforms FLASH-G by an average of 17.6 absolute percentage points (79.2% $\rightarrow$ 96.8%). Under identical representations, architectures, and hyperparameters, merely replacing the noise prior with the history polynomial yields this substantial gap, quantitatively validating the contribution of the FLASH mechanism detailed in Sec. 3.2.
- Advantage over the strongest baseline:** FLASH maintains a 2% to 8% lead (average 4.8%) over A2A-Noise. While A2A-Noise utilizes an informative prior within a discrete action space, FLASH elevates this prior to the polynomial coefficient space, raising the performance ceiling under the same inference budget.

These advantages are largely attributed to the smoother trajectories from sparse sampling: Outputs are constrained to smooth polynomials. The temporally downsampled H_{poly} window covers a vastly extended physical horizon (by a factor of $k = 4$), drastically mitigating action jitter and unexecutable commands, which is particularly crucial for long-horizon tasks like *Close Box* and *Pick-Place Bowl*. Moreover, position and velocity continuity at chunk junctions in training data are explicitly guaranteed by the KKT-constrained least squares (Eq. (11)), further alleviating switching jitter between adjacent polynomials.

D Training efficiency: remaining tasks

Fig. 8 shows the training curves for the two tasks not included in the main text. On *Close Box*, FLASH reaches 100% at 10k steps while most baselines plateau below 90%. On *Open Drawer*, FLASH converges to 92% at 10k steps, while strongest competitor at that point (A2A-Noise) lags by 10 pp.

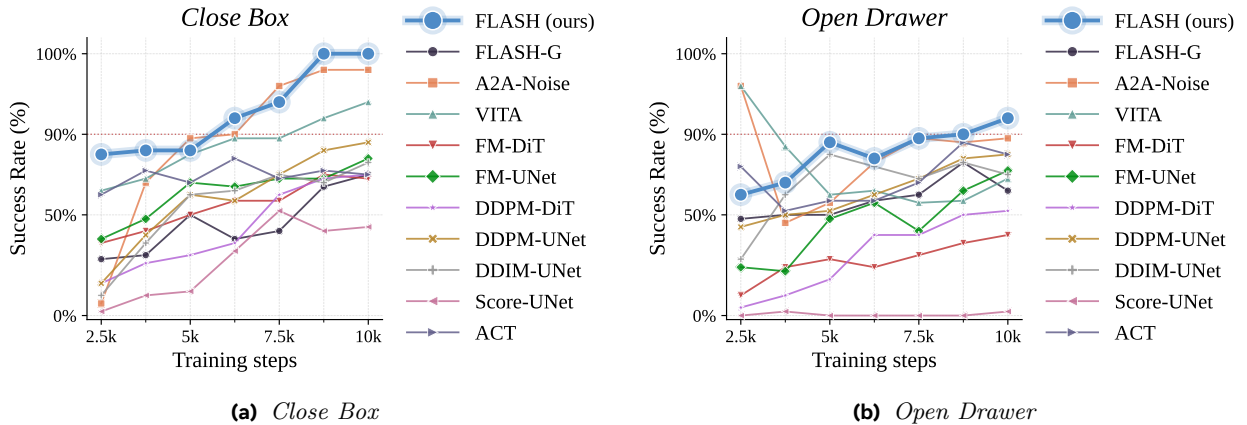


Figure 8 Training efficiency on remaining tasks. Same protocol as Fig. 4.

E Inference speed: additional metrics

This section provides the two additional inference speed metrics summarized in Section 4.2.

Per-call latency (Average Inference Time). As shown in Fig. 9, this metric is defined as the amortized inference time per environmental step (total inference time divided by total environmental steps). It fully amortizes chunking and NFE, remains independent of success rates or episode lengths. FLASH ranks first at 0.32 ms:

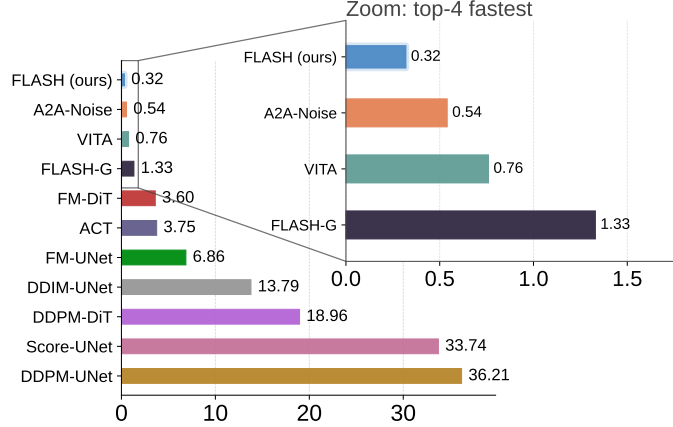


Figure 9 Per-call latency (Average Inference Time) on simulated Stack Cube.

- $113\times$ faster than DDPM-UNet (36.21 ms);
- $1.7\times$ faster than the strongest $N_{\text{NFE}} = 1$ baseline, A2A-Noise (0.54 ms);
- $4.2\times$ faster than FLASH-G (1.33 ms), which shares the same polynomials but starts from noise;
- $16\times$ faster than the median of the 10 baselines (~ 5 ms).

Real-world task wall time. Fig. 10 presents execution snapshots of the real-world *Place Cube* task and compares the wall-clock time across 6 representative policies over 5 rollouts. Real-world latency encompasses the full-pipeline cost of computation, interpolation/communication, and mechanical response, imposing stricter constraints on real-time capabilities than simulation. FLASH completes the task in 8.00 ± 0.24 s on average, the fastest among the 6 policies:

- $2.15\times$ faster than DDPM-UNet (17.29 s) and DDPM-DiT (17.00 s);
- $1.29\times$ faster than the homologous FLASH-G (10.34 s);
- Saves 45% of wall-clock time compared to the baseline average of 14.53 s.

Notably, the standard deviation of FLASH across 5 rollouts is only 0.24 s, an order of magnitude ($13\times$) smaller than that of DDPM-DiT (± 3.21 s), quantitatively reflecting the high execution repeatability of polynomial outputs in the physical world.

Mechanistic breakdown of speed advantages We decompose FLASH’s systematic speed advantage into two independent mechanistic contributions, rigorously isolated through pairwise comparisons among FLASH, FLASH-G, and A2A-Noise:

(i) *Sparse sampling training $\rightarrow$ each inference covers $4\times$ horizon.* Both FLASH and FLASH-G utilize the sparse sampling polynomial training strategy (with a temporal stride $k = 4$). A single forwardly generated polynomial covers $4\times$ the control duration of discrete action policies, significantly lowering the inference cost. FLASH-G (1.33 ms/call, 159 ms/episode), despite starting from noise, remains faster than almost all baselines except A2A-Noise and VITA, serving as independent empirical evidence for this mechanism.

(ii) *Flow matching initiated from history polynomials.* Although FLASH-G benefits from sparse sampling, its flow still starts from pure *Gaussian* noise and requires multi-step ODE integration. This explains why it is slower than A2A-Noise and VITA. These two baselines both employ an informative prior, indicating that the information content of the starting point intrinsically bounds the speed ceiling. FLASH fuses both mechanisms: sparse sampling + history polynomial starting point, surpassing A2A-Noise by $1.7\times$ at 0.32 ms. FLASH’s $4.2\times$ (per call) and $5.1\times$ (per episode) speedups over FLASH-G stem entirely from the difference in

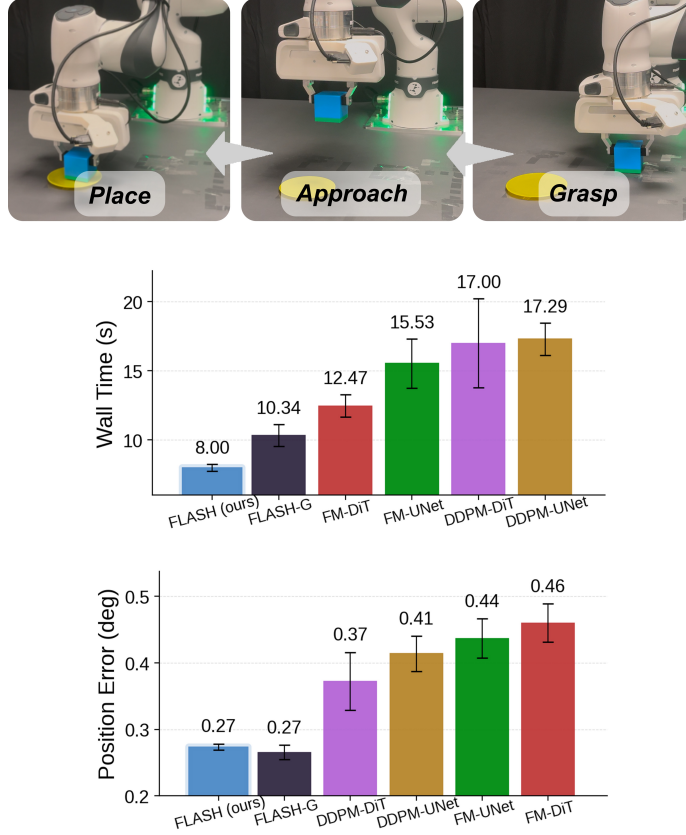


Figure 10 Real-world Place Cube. Top: Execution sequence. Middle: Task completion wall time. Bottom: Tracking error.

the flow’s starting point, quantitatively proving the independent contribution of the history-anchored flow mechanism to inference speed.

F Trajectory smoothness and tracking error: detailed analysis

This section provides the full analyses summarized in Section 4.3.

Time-series tracking error (full analysis). In Fig. 5 (middle), we compute the sum of absolute tracking errors across the 7 arm joints (J3 to J9), given by $\sum_{j=3}^9 |e_j(t)|$. Both the mean and peak errors of FLASH are substantially lower than those of FM-DiT (a mean gap of $4.70\times$ and a peak gap of $3.18\times$). FM-DiT’s error curve exhibits spikes at every chunk boundary, a direct consequence of the inter-chunk jitter inherent in discrete action policies. Additionally, FLASH completes the task in just 0.84 s (FM-DiT takes 1.22 s, $1.45\times$ slower), achieving simultaneous compression in both the “duration” and “magnitude” dimensions of the error.

Cumulative tracking error IAE. We also employ the Integral of Absolute Error (IAE), a standard metric in control theory defined as $IAE = \int \sum_{j=3}^9 |e_j(t)| dt$ (7 joints / unit: $\text{deg} \cdot \text{s}$), to aggregate the compound cost of “duration $\times$ magnitude”. For the same *Pick Cube* rollout, FLASH achieves 1.80 $\text{deg} \cdot \text{s}$, while FM-DiT scores 12.23 ($6.80\times$ FLASH).

Real-world control stack (Fig. 10(bottom)). We operate the Franka robot in **joint torque control** mode, where the host directly computes and sends torque signals to the Franka, rather than relying on Franka’s internal dynamic solver. This ensures that tracking errors and practical control frequencies directly reflect the policy’s output quality, unmasked by internal engineering smoothing techniques. Notably, due to the inherent properties of continuous polynomials, FLASH and FLASH-G can output signals directly at 1000 Hz, whereas

the other 4 baselines must rely on 10→1000 Hz linear interpolation to perform control. Moreover, FLASH and FLASH-G analytically provide the desired velocity $\dot{q}_d$ via polynomial differentiation (Eq. 4), directly supplying the torque controller’s velocity feed-forward term; discrete-action baselines lack this capability and must resort to noisy numerical differentiation, degrading tracking precision. This setup equally applies to the simulated experiments.

We measure the control-level 7-joint average tracking MAE over 6 policies $\times$ 5 rollouts. FLASH (0.274 ± 0.004 deg) and FLASH-G (0.266 ± 0.011 deg) perform virtually identically (the 0.008 deg mean difference is smaller than their respective inter-rollout standard deviations, rendering them statistically indistinguishable). DDPM-DiT, DDPM-UNet, FM-UNet, and FM-DiT yield 0.373, 0.414, 0.437, and 0.460 deg, respectively (representing a $1.36\times$ to $1.68\times$ error increase relative to FLASH). The inter-rollout standard deviation of FLASH is only ± 0.004 deg, the smallest among all policies and an order of magnitude lower than DDPM-DiT’s ± 0.043 deg, reflecting the high execution repeatability of polynomial trajectories on real hardware.

High-precision real-world Insert Cube task (full analysis). We validate FLASH under strict high-precision constraints on the real-world *Insert Cube* task (a precision insertion with millimeter-level tolerance). All models are trained with 10k optimizer steps. The first three subfigures of Fig. 6 chronologically illustrate the execution process (grasp $\rightarrow$ approach $\rightarrow$ insert). The fourth subfigure summarizes the real-world success rates of 8 policies using a radar chart: FLASH ranks first with 100%, leading the other 7 baselines by an average of 47%. Notably, while DDPM-UNet performs adequately in simulation (84%), its success rate plummets to only 10% on the real robot, highlighting the fragility of discrete action diffusion policies under high-precision physical constraints.

Across both simulated (Fig. 5, middle) and real-world results (Fig. 10 (bottom), Fig. 6), FLASH is consistently superior to non-polynomial baselines. This confirms that control-level tracking precision is jointly improved by three advantages exclusive to polynomial parameterization (the independent contribution of Section 3.1): output trajectory smoothness, native high-frequency control signal generation, and analytic velocity feed-forward for torque control. These three factors are orthogonal to the flow starting point (the FLASH-exclusive mechanism of Sec. 3.2).

G Post-hoc execution speed modulation

We empirically validate the post-hoc speed modulation property established in Section 3.1 on *Stack Cube* using the 10k step FLASH checkpoint. We fix the training stride $k_{\text{train}}=4$ and sweep the evaluation stride $k_{\text{eval}} \in \{1, \dots, 12, 16, 32\}$, which spans playback speeds from $4\times$ acceleration to $8\times$ deceleration relative to the training pace. Figure 5 (right) summarizes the resulting success rates.

FLASH maintains $\geq 93\%$ success across $k_{\text{eval}} \in \{3, 5, 6, 7, 8\}$, forming a wide high-SR plateau spanning $0.5\times$ to $1.33\times$ playback speed; notably, $k_{\text{eval}} \in \{5, 7\}$ surpass the training value $k_{\text{eval}}=4$ (98% vs. 96%), as a slightly larger T supplies the controller with denser intermediate commands, more than offsetting the mild observation distribution shift. The two failure modes are asymmetric: on the acceleration side, the most extreme $4\times$ speed-up still yields a non-zero 26% success rate; on the deceleration side, the collapse is sharper ($\geq 2.5\times$ slow-down fails outright), because over-stretched observation intervals leave the robot displaced too far between consecutive inferences.

H Ablation study details

H.1 Sparse sampling stride k

We sweep the temporal stride k defined in Section 3.1 from 1 to 18 on the simulated *Stack Cube* task with 400 demonstrations, fixing the *Legendre* degree at $K=6$, and evaluate 12 checkpoints every 1,250 training steps. Fig. 7 (left) reports the result.

- Necessity of sparse sampling.** Without sparse sampling ($k=1$) the success rate caps at 24% throughout training; raising k to 4 lifts it to 96%, a 72 pp gap that quantitatively justifies the strategy.

•**Bell-shaped optimum.** A sweet spot emerges at $k \in [3, 8]$ where every run converges to $\geq 93\%$, with $k = 8$ crossing 80% in only 5,000 steps (Fig. 7, left (a)). Beyond $k \geq 13$ performance collapses (52% at $k = 15$, 28% at $k = 18$). Two factors compound here: (i) the fixed-degree polynomial (Eq. 3) loses expressivity as the physical span $k \cdot H_{\text{poly}}$ grows; (ii) a single inference covers too much of the task that only ~ 2.7 forward passes per successful episode remain at $k = 18$, leaving the policy little chance to revise its plan from fresh visual feedback.

•**Pareto trade-off and practical speed limit.** Per-call latency falls monotonically from 0.97 ms ($k = 1$) to 0.15 ms ($k = 14$), and per-episode total inference time tracks it from 116 ms to 14.5 ms. Pushing k from 4 to 8 retains 98% success while halving both metrics to 0.21 ms / 20.3 ms. We retain $k = 4$ as the main-paper default, sweet-spot accuracy paired with a $3.7\times$ speed-up over $k = 1$, and treat $k = 8$ as a more aggressive setting whenever inference budget is the binding constraint.

H.2 Fit padding and cross-horizon continuity

We isolate the contribution of the two-tier extension introduced in Section 3.1 on the *Close Box* task, sharing identical training pipelines across the three configurations (last-4-checkpoint average success rate reported in Table 2).

Table 2 Ablation of fit padding and KKT continuity on *Close Box*.

Configuration	Success Rate (%)
$\delta = 0$, no KKT continuity	75.0
$\delta = 1$, no KKT continuity	86.0 (+11.0)
$\delta = 1 + \text{KKT continuity}$ (default)	97.5 (+11.5)

Fit padding pulls the constraint anchors into the stable interior of the *Legendre* fitting window and yields an 11.0 pp gain; stacking the KKT C^1 correction (Eq. 11) on top contributes another 11.5 pp, jointly closing a 22.5 pp gap to a near-saturated 97.5%. Both extensions are therefore necessary for the polynomial pipeline.

H.3 Number of function evaluations N_{NFE}

We sweep the inference $N_{\text{NFE}} \in \{1, 2, 4, \dots, 60\}$ on a single *Close Box* checkpoint trained for 2,500 steps; Fig. 7 (middle) reports success rate and per-call latency.

The single-step Euler ($N_{\text{NFE}} = 1$) reaches 80% success rate, *outperforming* the multi-step integrations at $N_{\text{NFE}} = 2$ (54%), 4 (56%), and 6 (68%), and matching the saturation level reached by $N_{\text{NFE}} \geq 10$ (mostly 86% to 94%, peaking at 96%). This counter-intuitive pattern directly validates the consistency loss $\mathcal{L}_{\text{cons}}$ (Eq. 8): by explicitly supervising the $\tau=0$ one-shot Euler update at training time, the model is specialized for the $N_{\text{NFE}} = 1$ regime, whereas small intermediate N_{NFE} values lose this specialization without yet accumulating enough multi-step accuracy. Meanwhile, per-call latency rises nearly linearly with N_{NFE} , from 0.32 ms at $N_{\text{NFE}} = 1$ to 3.56 ms at $N_{\text{NFE}} = 60$ (11 $\times$), so any $N_{\text{NFE}} > 1$ is a strict efficiency loss with no accuracy upside, justifying our deployment choice of single-step Euler integration.

H.4 Velocity feed-forward

FLASH derives the desired velocity signal analytically from the polynomial coefficients (Eq. 4), feeding it directly to the torque controller as a feed-forward term. To isolate the contribution of this component, we ablate the velocity feed-forward by sending only position commands (i.e., setting $\hat{\mathbf{v}} = \mathbf{0}$) while keeping all other components unchanged: the polynomial representation, sparse sampling, FLASH flow matching, and KKT continuity correction.

Experimental setup. We evaluate on the same 9 checkpoints (spanning 3 tasks $\times$ 3 training seeds) used in the preceding ablation, with 50 rollouts per checkpoint ($N = 450$ total). For each rollout, we record the mean absolute error (MAE, in degrees) between the commanded and actual joint positions across all timesteps, then compare the *with-velocity* and *without-velocity* conditions in a pairwise fashion.

Results (Fig. 7, right). Removing the analytic velocity feed-forward inflates the J7 (wrist) position-tracking MAE from 1.33° to 2.90° , a $2.18\times$ degradation. We report three complementary statistical measures to characterize this effect:

- **Paired t -test ($p < 10^{-100}$):** Because each rollout is executed under both conditions on the same checkpoint, we use a *paired* (dependent-samples) t -test rather than an independent two-sample test. The resulting p -value is astronomically small, indicating that the observed MAE increase is far beyond what random variation could explain.
- **Cohen’s $d = 1.78$:** While p -values indicate *whether* an effect exists, Cohen’s d quantifies *how large* it is by expressing the mean difference in units of the pooled standard deviation. By conventional benchmarks (Cohen, 2013), $d \geq 0.8$ is already considered a “large” effect; the observed $d = 1.78$ is more than twice that threshold, confirming that the velocity feed-forward produces a practically significant (not merely statistically significant) improvement.
- **Sample size ($N = 450$):** Reporting the sample size is essential for interpreting both the p -value and Cohen’s d . With 450 paired observations, the test has ample statistical power to detect even moderate effects; the extremely large d and vanishingly small p together rule out both Type I and Type II error concerns.

Six of seven arm joints exhibit the same pattern: the position-only controller consistently produces larger tracking errors, with the degradation most pronounced in the distal joints (J6, J7) where inertia is lower and high-frequency feed-forward compensation matters most. This confirms that the analytic velocity feed-forward is not merely a theoretical convenience but a critical component for precision torque control.